

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN  
A DISTANCIA

**TRABAJO PRÁCTICO INTEGRADOR**  
**Gestión de Datos de Países en Python**  
*Filtros, ordenamientos y estadísticas*

**Materia:** Programación 1

**Carrera:** Tecnicatura Universitaria en Programación

**Institución:** Universidad Tecnológica Nacional

**Estudiantes:** Mariano Gallo– Mariano Ezequiel Maidana

**Comisión:** Gallo (comisión 10) – Maidana (comisión 08)

**Docente Coordinador:** Alberto Cortez

**Docentes de la Materia:** Ariel Enferrel, Martín A. García, Cinthia Rigoni

**Docente/Tutor:** Luciano Chirolí

## Índice

|                                       |    |
|---------------------------------------|----|
| 1. Introducción                       | 3  |
| 2. Objetivos del trabajo              | 3  |
| 3. Descripción de la consigna         | 4  |
| 4. Marco teórico                      | 4  |
| 5. Diseño del caso práctico           | 7  |
| 6. Decisiones técnicas y arquitectura | 7  |
| 7. Funcionalidades implementadas      | 13 |
| 8. Funcionalidades implementadas      | 14 |
| 9. Pruebas y resultados obtenidos     | 23 |
| 10. Conclusiones                      | 24 |
| 11. Bibliografía / Webgrafía          | 25 |
| 12. Links del proyecto                | 25 |

## 1. Introducción

El presente Trabajo Práctico Integrador desarrolla una aplicación de consola en Python orientada a la gestión de datos de países. El sistema permite cargar información desde un archivo CSV, almacenar los registros en memoria mediante estructuras de datos, consultar países, aplicar filtros, ordenar resultados y generar estadísticas básicas a partir del conjunto de datos.

El proyecto integra contenidos fundamentales de Programación 1, tales como listas, diccionarios, funciones, estructuras condicionales y repetitivas, lectura de archivos, validación de datos, ordenamientos y cálculos estadísticos simples. A través de este desarrollo se busca aplicar estos conceptos en un caso práctico concreto, simulando un pequeño sistema de consulta y análisis de información.

La elección de un dataset de países permite trabajar con datos reales o verosímiles, compuestos por nombre, población, superficie y continente. Estos campos facilitan la implementación de búsquedas, filtros por rangos y estadísticas comparativas, permitiendo observar de manera clara la utilidad de las estructuras de datos y de la modularización del código.

## 2. Objetivos del trabajo

El objetivo general del proyecto es desarrollar una aplicación en Python que permita gestionar información de países desde un archivo CSV, aplicando los conceptos principales vistos en la materia Programación 1.

- Representar cada país mediante un diccionario con los campos nombre, población, superficie y continente.
- Almacenar el conjunto de países en una lista para poder recorrer, consultar y modificar los registros.
- Leer los datos iniciales desde un archivo CSV incluido en el repositorio del proyecto.
- Implementar un menú interactivo en consola para acceder a las funcionalidades del sistema.
- Agregar y actualizar datos validando que no existan campos vacíos ni valores numéricos inválidos.
- Buscar países por coincidencia parcial o exacta del nombre.
- Filtrar países por continente, rango de población y rango de superficie.
- Ordenar los países por nombre, población o superficie en forma ascendente o descendente.
- Calcular estadísticas básicas como mayor y menor población, promedios y cantidad por continente.
- Organizar el código en funciones claras, donde cada una cumpla una responsabilidad específica.

### 3. Descripción de la consigna

La consigna solicita desarrollar una aplicación en Python para gestionar información sobre países. Cada país debe incluir nombre, población, superficie en kilómetros cuadrados y continente. El sistema debe leer datos desde un archivo CSV, ofrecer un menú en consola y permitir operaciones de alta, actualización, búsqueda, filtrado, ordenamiento y estadísticas.

Además del código fuente, el trabajo requiere un repositorio en GitHub, un archivo CSV base, un README con instrucciones de uso.

### 4. Marco teórico

En esta sección se describen los conceptos principales aplicados en el desarrollo del sistema y su relación directa con el programa implementado.

#### 4.1 Listas

Una lista en Python es una estructura de datos que permite almacenar varios elementos bajo un mismo nombre. Es una colección ordenada y mutable, es decir, sus elementos pueden agregarse, eliminarse o modificarse durante la ejecución del programa. En el proyecto, la lista principal se denomina `países` y contiene todos los registros cargados desde el archivo CSV.

El uso de una lista es adecuado porque el sistema necesita recorrer todos los países para mostrarlos, buscarlos, filtrarlos, ordenarlos y calcular estadísticas. Por ejemplo, cuando se desea encontrar el país con mayor población, el programa recorre la lista completa comparando el valor de población de cada diccionario.

#### 4.2 Diccionarios

Un diccionario es una estructura que almacena datos en pares clave-valor. Cada clave identifica un dato específico y permite acceder a él de manera clara. En este proyecto, cada país se representa mediante un diccionario con las claves `nombre`, `poblacion`, `superficie` y `continente`.

Esta elección permite modelar cada registro de forma ordenada. Por ejemplo, para acceder a la población de Argentina, el programa utiliza la clave `pais["poblacion"]`. De esta manera, el código resulta más legible que si se utilizaran posiciones numéricas dentro de una lista.

#### 4.3 Funciones

Las funciones permiten dividir un programa en bloques de código reutilizables. Cada función recibe datos, realiza una tarea específica y puede devolver un resultado. En el proyecto se utilizaron funciones para cargar el CSV, mostrar el menú, validar entradas, agregar países, actualizar datos, buscar, filtrar, ordenar y calcular estadísticas.

La modularización facilita la lectura y el mantenimiento del programa. Por ejemplo, la función `pedir_entero_positivo` se reutiliza cada vez que se necesita solicitar una población, una superficie o un valor numérico para un rango. Esto evita repetir código y reduce la posibilidad de errores.

#### 4.4 Condicionales

Las estructuras condicionales permiten que el programa tome decisiones según una condición. En Python se implementan principalmente con `if`, `elif` y `else`. En este sistema se utilizan para validar opciones del menú, controlar errores de entrada, verificar si una búsqueda tiene resultados y decidir qué criterio de ordenamiento aplicar.

Por ejemplo, si el usuario elige la opción 4 del menú, el programa ejecuta la función de búsqueda por nombre. Si ingresa una opción inexistente, se muestra un mensaje de error y el menú vuelve a presentarse.

#### 4.5 Estructuras repetitivas

Las estructuras repetitivas permiten ejecutar un bloque de código varias veces. En el proyecto se utilizan ciclos `while` y `for`. El ciclo `while` mantiene activo el menú hasta que el usuario selecciona la opción de salir. El ciclo `for` se utiliza para recorrer la lista de países en búsquedas, filtros, ordenamientos auxiliares y estadísticas.

El uso de ciclos es fundamental porque la cantidad de países puede variar. Si se agregan nuevos registros al CSV, el programa puede recorrerlos automáticamente sin necesidad de modificar el código fuente.

#### 4.6 Archivos CSV

CSV significa Comma-Separated Values, es decir, valores separados por comas. Es un formato simple utilizado para almacenar datos tabulares en texto plano. En este trabajo, el archivo `países.csv` contiene una primera línea de encabezados y luego una línea por cada país.

El programa lee el CSV al iniciar y transforma cada fila en un diccionario. De esta forma, el archivo funciona como fuente de datos inicial y también como medio de persistencia cuando se agregan o actualizan registros.

#### 4.7 Ordenamientos

Ordenar datos consiste en reorganizar una colección de elementos según un criterio determinado. En este proyecto se implementan ordenamientos por nombre, población y superficie. Además, se permite elegir si el orden será ascendente o descendente.

Para realizar el ordenamiento se utiliza la función `sorted`, indicando una función auxiliar como criterio. Por ejemplo, `obtener_poblacion` devuelve el valor de población de cada país, permitiendo ordenar la lista según ese campo.

#### **4.8 Estadísticas básicas**

Las estadísticas básicas permiten resumir información del conjunto de datos. En el sistema se calculan indicadores simples pero útiles: país con mayor población, país con menor población, promedio de población, promedio de superficie y cantidad de países por continente.

Estos cálculos muestran cómo las estructuras de datos pueden utilizarse no solo para almacenar información, sino también para obtener conclusiones a partir de los registros cargados.

## 5. Diseño del caso práctico

El diseño del sistema parte de una estructura simple: un archivo CSV vacío como origen de datos, una lista de diccionarios como estructura principal en memoria y un menú de consola como interfaz de usuario. El flujo general consiste en cargar los datos, mostrar opciones, ejecutar la operación seleccionada y volver al menú hasta que el usuario decida finalizar el programa.

### Diagrama de flujo general del sistema

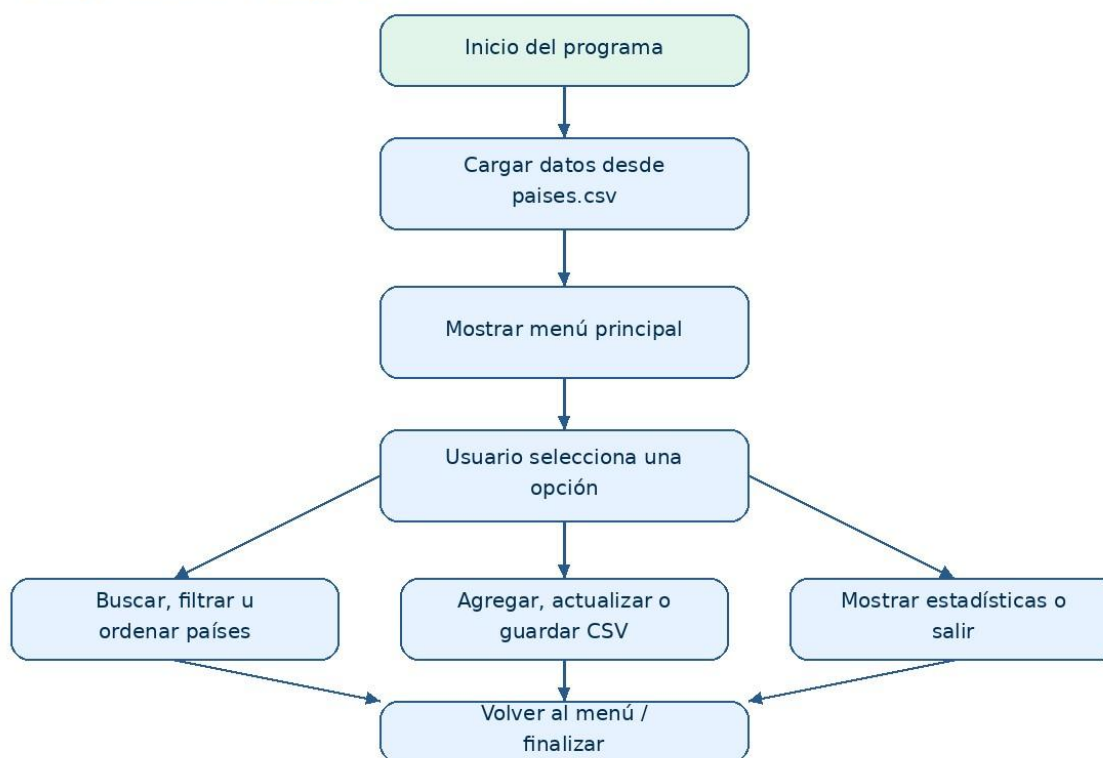


Figura 1. Diagrama de flujo general del sistema.

El flujo propuesto permite mantener el programa organizado. La carga inicial se realiza una sola vez al comenzar. Luego, cada funcionalidad opera sobre la lista de países. En los casos de agregado o actualización, los cambios se guardan nuevamente en el archivo CSV para conservar la información.

## 6. Decisiones técnicas y arquitectura

### 6.1 Estructura del proyecto

El proyecto se organiza mediante una estructura modular, separando el código fuente en distintos archivos .py según la responsabilidad de cada grupo de funciones. Esta decisión permite mejorar la

organización, facilitar la lectura del código y cumplir con el criterio de modularización solicitado para el Trabajo Práctico Integrador.

La estructura general del proyecto es la siguiente:

| Archivo / Carpeta        | Descripción                                                                                                                                     |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| main.py                  | Archivo principal del sistema. Funciona como punto de entrada del programa, carga los datos iniciales desde el CSV y ejecuta el menú principal. |
| países.csv               | Archivo de datos base que contiene los registros iniciales de países con nombre, población, superficie y continente.                            |
| README.md                | Archivo de documentación breve del proyecto, con descripción del sistema, instrucciones de ejecución, ejemplos de uso e integrantes.            |
| .gitignore               | Archivo utilizado para excluir del repositorio elementos innecesarios.                                                                          |
| capturas/                | Carpeta opcional destinada a guardar evidencias de ejecución del programa y capturas utilizadas en el informe                                   |
| informe_tpi.pdf          | Documento académico y técnico del Trabajo Práctico Integrador.                                                                                  |
| modulos/                 | Carpeta que contiene los módulos auxiliares del sistema.                                                                                        |
| modulos/__init__.py      | Archivo vacío que permite reconocer la carpeta modulos como paquete de Python.                                                                  |
| modulos/datos.py         | Módulo encargado de la lectura y escritura del archivo CSV. Contiene las funciones relacionadas con la persistencia de datos.                   |
| modulos/validacion.py    | Módulo que agrupa las funciones utilizadas para validar entradas del usuario.                                                                   |
| modulos/visualización.py | Módulo destinado a mostrar información por consola, incluyendo el menú principal, los datos de un país y listados de países.                    |
| modulos/operaciones.py   | Módulo que contiene las operaciones principales del sistema: agregar, actualizar, buscar y filtrar países.                                      |

|                         |                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------|
| modulos/ordenamiento.py | Módulo encargado de ordenar los países por nombre, población o superficie.                      |
| modulos/estadísticas.py | Módulo encargado de calcular y mostrar los indicadores estadísticos solicitados en la consigna. |

Esta organización permite que el archivo main.py no concentre toda la lógica del sistema, sino que coordine la ejecución general del programa mediante llamadas a funciones ubicadas en módulos auxiliares. De esta manera, el código resulta más claro, más mantenible y más fácil de corregir o ampliar.

## 6.2 Estructura de datos utilizada

La estructura principal del sistema es una lista llamada `países`. Cada elemento de esa lista es un diccionario que representa un país. Esta solución permite combinar dos estructuras vistas en la materia: la lista como colección de registros y el diccionario como representación clara de cada registro individual.

Cada diccionario almacena los datos de un país utilizando claves descriptivas. Esto facilita el acceso a cada valor y mejora la legibilidad del código, ya que se puede acceder directamente a datos como el nombre, la población, la superficie o el continente.

Ejemplo conceptual de la estructura utilizada:

| Clave      | Tipo de dato | Descripción                          |
|------------|--------------|--------------------------------------|
| nombre     | string       | Nombre del país.                     |
| poblacion  | int          | Cantidad de habitantes.              |
| superficie | int          | Superficie en kilómetros cuadrados.  |
| continente | string       | Continente al que pertenece el país. |

Ejemplo de un país representado como diccionario:

```
{
    "nombre": "Argentina",
    "poblacion": 45376763,
    "superficie": 2780400,
```

```
"continente": "América"
}
```

Ejemplo de la lista general de países:

```
países = [
    {
        "nombre": "Argentina",
        "poblacion": 45376763,
        "superficie": 2780400,
        "continente": "América"
    },
    {
        "nombre": "Japón",
        "poblacion": 125800000,
        "superficie": 377975,
        "continente": "Asia"
    }
]
```

Esta estructura fue elegida porque se adapta correctamente al dominio del problema. Cada país posee varios atributos relacionados entre sí, por lo que el diccionario permite agruparlos de manera ordenada. A su vez, la lista permite almacenar múltiples países y recorrerlos mediante estructuras repetitivas para realizar búsquedas, filtros, ordenamientos y cálculos estadísticos.

### 6.3 Modularización

El programa fue dividido en módulos funcionales para que cada archivo tenga una responsabilidad específica dentro del sistema. Esta decisión favorece la claridad del proyecto, evita concentrar todo el código en un único archivo y permite trabajar de forma más ordenada.

La modularización aplicada responde al criterio de separar el programa en partes lógicas: manejo de archivos, validaciones, visualización, operaciones principales, filtros, ordenamientos, estadísticas y función principal.

| Bloque | Funciones principales | Responsabilidad |
|--------|-----------------------|-----------------|
|--------|-----------------------|-----------------|

|                          |                                                                                                                                                                                          |                                                                                             |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| main.py                  | main                                                                                                                                                                                     | Ejecutar el programa, cargar los datos iniciales y coordinar las opciones del menú.         |
| modulos/datos.py         | cargar_paises_csv,<br>guardar_paises_csv                                                                                                                                                 | Leer los datos desde el archivo CSV y guardar los cambios realizados durante la ejecución.  |
| modulos/validacion.py    | pedir_texto_no_vacio,<br>pedir_nombre_pais,<br>pedir_entero_positivo,<br>pedir_rango, pedir_continente                                                                                   | Controlar que las entradas del usuario sean válidas y evitar errores por datos incorrectos. |
| modulos/visualizacion.py | mostrar_menu,<br>mostrar_pais,<br>mostrar_paises                                                                                                                                         | Presentar información en consola de forma clara y ordenada.                                 |
| modulos/operaciones.py   | agregar_pais, actualizar_pais,<br>buscar_pais_por_nombre,<br>filtrar_por_continente,<br>filtrar_por_poblacion,<br>filtrar_por_superficie                                                 | Gestionar las acciones principales del sistema sobre la lista de países.                    |
| modulos/estadistica.py   | mostrar_pais_mayor_poblacion,<br>mostrar_pais_menor_poblacion,<br>calcular_promedio_poblacion,<br>calcular_promedio_superficie,<br>contar_paises_por_continente,<br>mostrar_estadisticas | Calcular y mostrar indicadores estadísticos del dataset.                                    |
| modulos/ordenamiento.py  | obtener_nombre,<br>obtener_poblacion,<br>obtener_superficie,<br>ordenar_paises                                                                                                           | Ordenar los países por distintos criterios, de forma ascendente o descendente.              |

Esta separación permite que cada función tenga una única responsabilidad. Por ejemplo, las funciones de validación no muestran estadísticas ni modifican datos; solamente controlan las entradas. Del mismo modo, las funciones de estadísticas no se ocupan de leer archivos ni de mostrar el menú, sino de calcular indicadores a partir de los datos cargados.

El archivo main.py funciona como coordinador general. Desde allí se importan las funciones necesarias de los demás módulos y se ejecuta el ciclo principal del menú. Cuando el usuario selecciona una opción, main.py llama a la función correspondiente ubicada en el módulo adecuado.

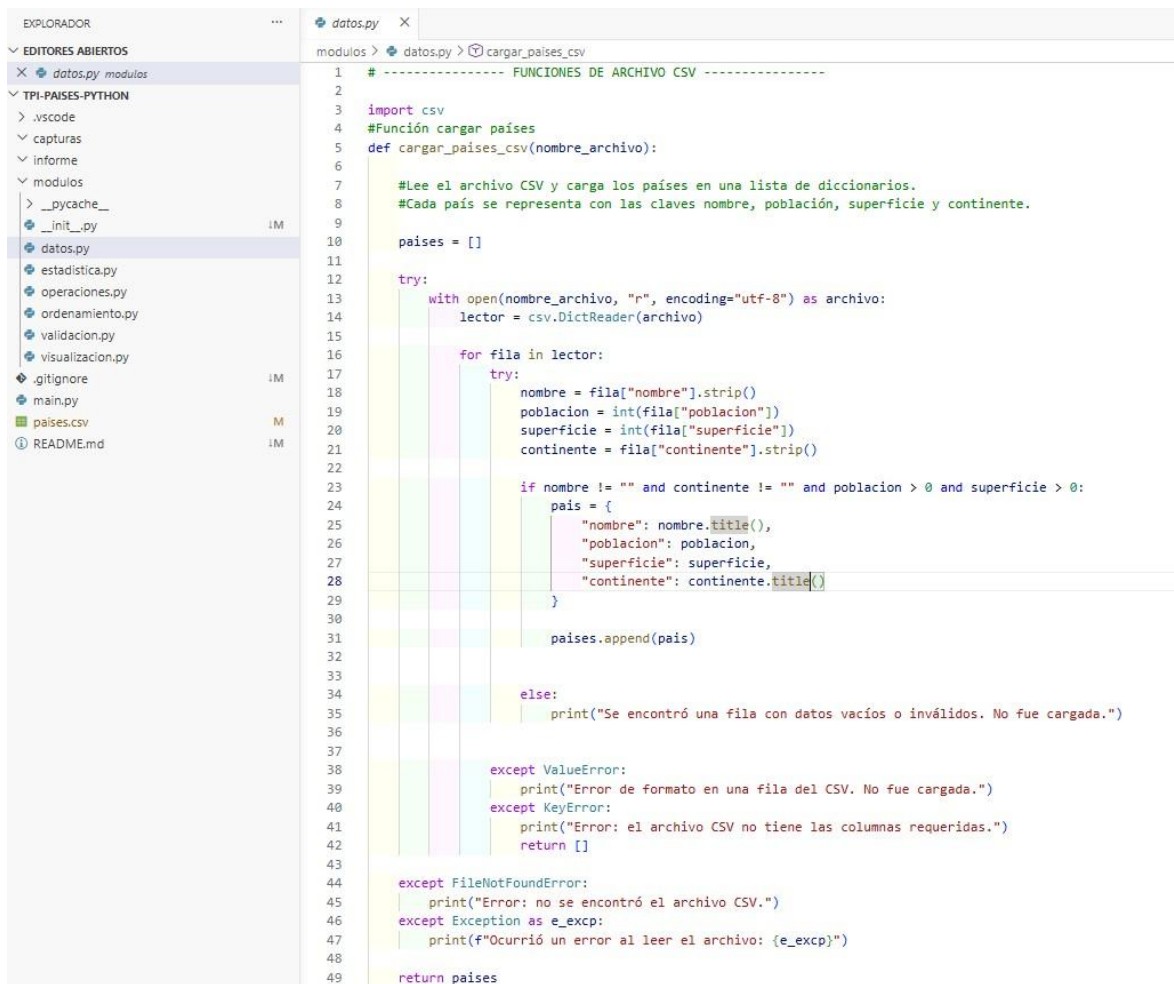
Esta arquitectura permite cumplir con una modularización más clara y facilita futuras modificaciones. Por ejemplo, si se desea mejorar la forma de guardar los datos, se puede modificar el módulo datos.py sin alterar las funciones de búsqueda, filtrado o estadísticas. De la misma manera, si se desea

cambiar la presentación en consola, se puede trabajar sobre `visualizacion.py` sin afectar la lógica principal del sistema.

## 6.4 Capturas del código

A continuación, se incluyen capturas de fragmentos representativos del código fuente implementado.

La primera evidencia muestra la carga del CSV y la conversión de datos a la estructura interna.



```

1  # ----- FUNCIONES DE ARCHIVO CSV -----
2
3  import csv
4  #Función cargar países
5  def cargar_paises_csv(nombre_archivo):
6
7      #Lee el archivo CSV y carga los países en una lista de diccionarios.
8      #Cada país se representa con las claves nombre, población, superficie y continente.
9
10     paises = []
11
12     try:
13         with open(nombre_archivo, "r", encoding="utf-8") as archivo:
14             lector = csv.DictReader(archivo)
15
16             for fila in lector:
17                 try:
18                     nombre = fila["nombre"].strip()
19                     poblacion = int(fila["poblacion"])
20                     superficie = int(fila["superficie"])
21                     continente = fila["continente"].strip()
22
23                     if nombre != "" and continente != "" and poblacion > 0 and superficie > 0:
24                         pais = {
25                             "nombre": nombre.title(),
26                             "poblacion": poblacion,
27                             "superficie": superficie,
28                             "continente": continente.title()
29                         }
30
31                         paises.append(pais)
32
33                     else:
34                         print("Se encontró una fila con datos vacíos o inválidos. No fue cargada.")
35
36                 except ValueError:
37                     print("Error de formato en una fila del CSV. No fue cargada.")
38                 except KeyError:
39                     print("Error: el archivo CSV no tiene las columnas requeridas.")
40                 return []
41
42     except FileNotFoundError:
43         print("Error: no se encontró el archivo CSV.")
44     except Exception as e_excp:
45         print(f"Ocurrió un error al leer el archivo: {e_excp}")
46
47     return paises
48
49

```

Figura 2. Fragmento de código para cargar datos desde el archivo CSV.

La segunda evidencia muestra la función principal, donde se carga el dataset y se controla el menú del sistema mediante condicionales.

```

7      filtrar_por_continente,
8      filtrar_por_poblacion,
9      filtrar_por_superficie,
10     )
11     from modulos.estadistica import mostrar_estadisticas
12     from modulos.ordenamiento import ordenar_paises
13
14
15     archivo_csv = "países.csv"
16
17     #Función principal del programa, que carga los datos, muestra el menú y ejecuta las opciones seleccionadas por el usuario hasta que decida salir.
18     def main():
19         paises = cargar_paises_csv(archivo_csv)
20
21         opcion = ""
22
23         while opcion != "0":
24             mostrar_menu()
25             opcion = input("Ingrese una opción: ")
26
27             if opcion == "1":
28                 mostrar_paises(paises)
29
30             elif opcion == "2":
31                 agregar_pais(paises, archivo_csv)
32
33             elif opcion == "3":
34                 actualizar_pais(paises, archivo_csv)
35
36             elif opcion == "4":
37                 buscar_pais_por_nombre(paises)
38
39             elif opcion == "5":
40                 filtrar_por_continente(paises)
41
42             elif opcion == "6":
43                 filtrar_por_poblacion(paises)
44
45             elif opcion == "7":
46                 filtrar_por_superficie(paises)
47
48             elif opcion == "8":
49                 ordenar_paises(paises)
50
51             elif opcion == "9":
52                 mostrar_estadisticas(paises)
53
54             elif opcion == "0":
55                 print("Saliendo del programa...")
56
57             else:
58                 print("Opción inválida. Intente nuevamente.")
59
60
61     main()

```

Figura 3. Fragmento de código correspondiente al menú principal.

## 7. Funcionalidades implementadas

El sistema implementa las funcionalidades mínimas solicitadas en la consigna. Todas ellas se encuentran disponibles desde el menú principal de consola.

| Funcionalidad          | Descripción                                                                        | Función asociada       |
|------------------------|------------------------------------------------------------------------------------|------------------------|
| Mostrar países         | Lista todos los registros cargados desde el CSV.                                   | mostrar_paises         |
| Agregar país           | Solicita nombre, población, superficie y continente; valida datos y guarda el CSV. | agregar_pais           |
| Actualizar país        | Busca un país por nombre exacto y modifica población y superficie.                 | actualizar_pais        |
| Buscar por nombre      | Permite coincidencia parcial o exacta sin diferenciar mayúsculas.                  | buscar_pais_por_nombre |
| Filtrar por continente | Muestra países pertenecientes a un continente ingresado.                           | filtrar_por_continente |
| Filtrar por población  | Muestra países dentro de un rango de población.                                    | filtrar_por_poblacion  |
| Filtrar por superficie | Muestra países dentro de un rango de superficie.                                   | filtrar_por_superficie |

|                |                                                                              |                      |
|----------------|------------------------------------------------------------------------------|----------------------|
| Ordenar países | Ordena por nombre, población o superficie en forma ascendente o descendente. | ordenar_paises       |
| Estadísticas   | Calcula mayor y menor población, promedios y cantidad por continente.        | mostrar_estadisticas |

### 7.1 Validaciones y manejo de errores

El programa incorpora validaciones para evitar campos vacíos, números negativos, valores no enteros, rangos inválidos, nombres de países compuestos únicamente por números y continentes escritos de forma incorrecta. También se controla la lectura del archivo CSV mediante bloques tryexcept, evitando que el programa finalice abruptamente ante errores de formato, ausencia del archivo o columnas incorrectas.

Una mejora aplicada fue la validación del continente, permitiendo ingresar opciones con o sin tilde. Por ejemplo, el usuario puede escribir America o América, y el sistema lo interpreta correctamente como América. Esto mejora la experiencia de uso y evita rechazar entradas válidas por diferencias ortográficas menores.

También se mejoró la validación del ordenamiento. Si el usuario ingresa una opción inválida al seleccionar el criterio de ordenamiento, el sistema muestra un mensaje de error y vuelve al menú principal sin solicitar innecesariamente el tipo de orden ascendente o descendente.

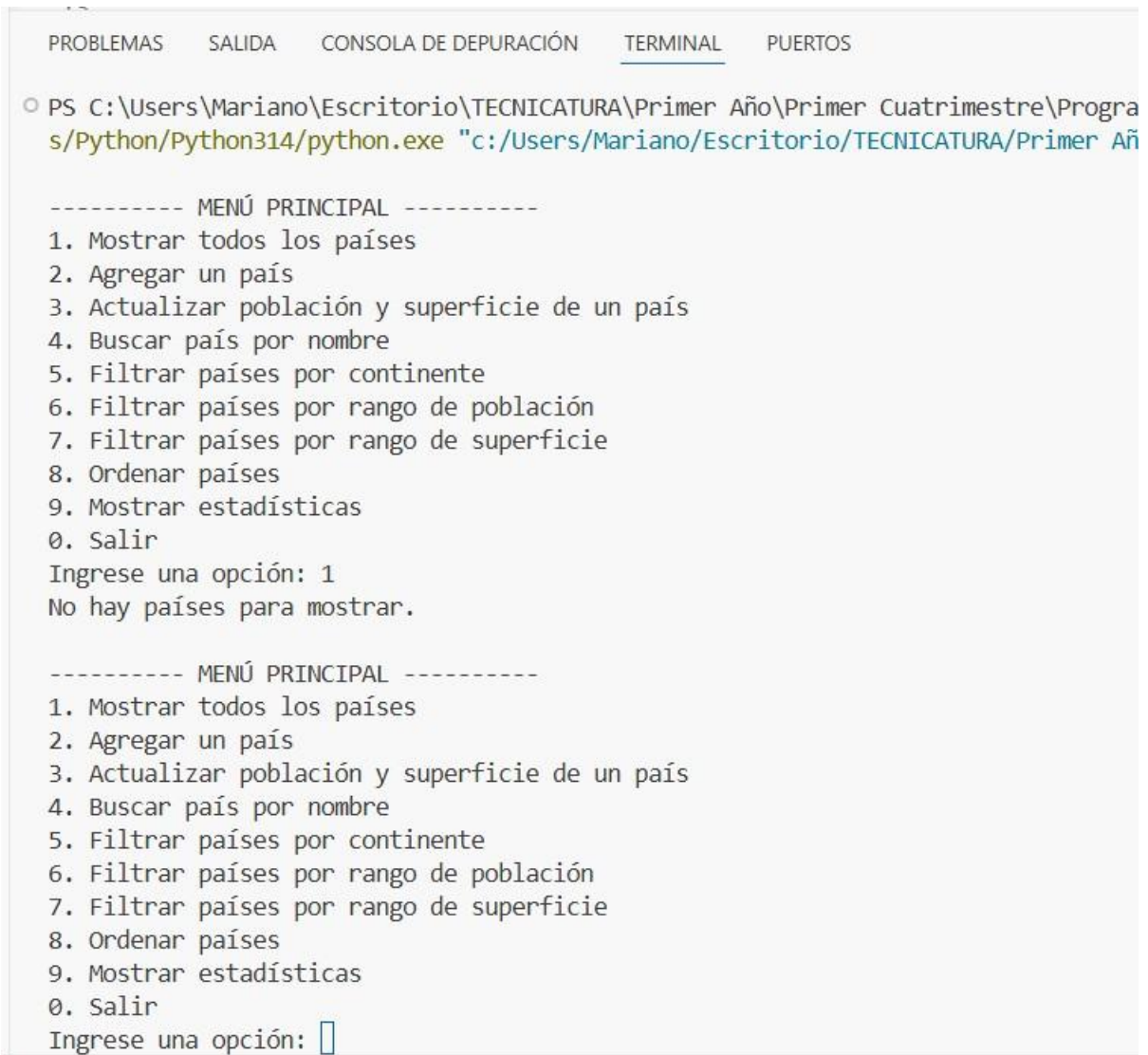
Estas validaciones son importantes porque hacen que el sistema sea más robusto frente a entradas inesperadas y permiten cumplir con el objetivo de brindar mensajes claros de éxito o error.

## 8. Pruebas y resultados obtenidos

Para verificar el funcionamiento del programa se realizaron pruebas sobre las opciones principales del menú. Como el archivo paises.csv se entrega inicialmente vacío, pero con encabezado, las pruebas se realizaron agregando registros desde el propio sistema y luego utilizando esos datos para consultar, buscar, filtrar, ordenar y calcular estadísticas.

### 8.1 Menú principal y carga inicial

La primera prueba consistió en ejecutar el programa con el archivo paises.csv sin registros iniciales. El sistema inició correctamente, cargó una lista vacía y mostró el menú principal sin producir errores.



```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

○ PS C:\Users\Mariano\Escritorio\TECNICATURA\Primer Año\Primer Cuatrimestre\Progra
s/Python/Python314/python.exe "c:/Users/Mariano/Escritorio/TECNICATURA/Primer Año

----- MENÚ PRINCIPAL -----
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar país por nombre
5. Filtrar países por continente
6. Filtrar países por rango de población
7. Filtrar países por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
0. Salir
Ingrese una opción: 1
No hay países para mostrar.

----- MENÚ PRINCIPAL -----
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar país por nombre
5. Filtrar países por continente
6. Filtrar países por rango de población
7. Filtrar países por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
0. Salir
Ingrese una opción: █

```

Figura 4. Ejecución inicial del sistema con CSV vacío

## 8.2 Agregado de países

Luego se probó la opción de agregar país. El sistema solicitó nombre, población, superficie y continente. Se verificó que no aceptara campos vacíos, valores numéricos inválidos ni nombres compuestos únicamente por números. Una vez cargado un país válido, el sistema lo agregó a la lista y guardó los datos en el archivo CSV.

```

----- MENÚ PRINCIPAL -----
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar país por nombre
5. Filtrar países por continente
6. Filtrar países por rango de población
7. Filtrar países por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
0. Salir
Ingrese una opción: 2

----- AGREGAR PAÍS -----
Ingrese el nombre del país:
Error: el nombre del país no puede estar vacío.
Ingrese el nombre del país: 434
Error: el nombre del país no puede ser solo números.
Ingrese el nombre del país: Argentina
Ingrese la población: 1
Error: debe ingresar un número entero válido.
Ingrese la población: 0
Error: debe ingresar un número mayor a cero.
Ingrese la población: -4
Error: debe ingresar un número mayor a cero.
Ingrese la población: 432423
Ingrese la superficie en km²: 432432
Continentes disponibles:
America
Europa
Asia
Africa
Oceania
Ingrese el continente: America
Datos guardados correctamente en el archivo CSV.
País agregado correctamente.

```

*Figura 5. Carga de un país válido.*

### 8.3 Búsqueda por nombre

La búsqueda por nombre permite ingresar una coincidencia parcial o exacta. En esta prueba, el sistema devolvió los países cuyo nombre contenía el texto ingresado por el usuario, sin diferenciar mayúsculas y minúsculas.

```

----- MENÚ PRINCIPAL -----
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar país por nombre
5. Filtrar países por continente
6. Filtrar países por rango de población
7. Filtrar países por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
0. Salir
Ingrese una opción: 4

----- BUSCAR PAÍS -----
Ingrese el nombre o parte del nombre del país: ARgentina

Resultados encontrados:

----- LISTADO DE PAÍSES -----
Nombre: Argentina
Población: 432423
Superficie: 432432 km²
Continente: America
-----

```

Figura 6. Filtro de países por continente.

#### 8.4 Filtros por continente, población y superficie

Se probaron las opciones de filtrado disponibles en el menú. El filtro por continente permite ingresar un continente válido y obtener únicamente los países que pertenecen a ese grupo. Además, el sistema acepta continentes escritos de manera diferente a la mostrada como AMERICA, america AMERICA.

También se probaron los filtros por rango de población y por rango de superficie. En ambos casos, el programa solicita un valor mínimo y un valor máximo, valida que sean números enteros positivos y controla que el mínimo no sea mayor que el máximo.

```
----- MENÚ PRINCIPAL -----  
1. Mostrar todos los países  
2. Agregar un país  
3. Actualizar población y superficie de un país  
4. Buscar país por nombre  
5. Filtrar países por continente  
6. Filtrar países por rango de población  
7. Filtrar países por rango de superficie  
8. Ordenar países  
9. Mostrar estadísticas  
0. Salir  
Ingrese una opción: 5  
  
----- FILTRAR POR CONTINENTE -----  
Continentes disponibles:  
America  
Europa  
Asia  
Africa  
Oceania  
Ingrese el continente: america  
  
----- LISTADO DE PAÍSES -----  
Nombre: Argentina  
Población: 432423  
Superficie: 432432 km²  
Continente: America
```

Figura 7a. Filtro de países por continente.

```

----- MENÚ PRINCIPAL -----
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar país por nombre
5. Filtrar países por continente
6. Filtrar países por rango de población
7. Filtrar países por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
0. Salir
Ingrese una opción: 6

----- FILTRAR POR RANGO DE POBLACIÓN -----
Ingrese la población mínima: f
Error: debe ingresar un número entero válido.
Ingrese la población mínima: 0
Error: debe ingresar un número mayor a cero.
Ingrese la población mínima: -4
Error: debe ingresar un número mayor a cero.
Ingrese la población mínima: 34
Ingrese la población máxima: 6456456456

----- LISTADO DE PAÍSES -----
Nombre: Argentina
Población: 432423
Superficie: 432432 km²
Continente: America
-----

```

Figura 7b. Filtro de países por rango de población

```

----- MENÚ PRINCIPAL -----
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar país por nombre
5. Filtrar países por continente
6. Filtrar países por rango de población
7. Filtrar países por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
0. Salir
Ingrese una opción: 7

----- FILTRAR POR RANGO DE SUPERFICIE -----
Ingrese la superficie mínima en km²: f
Error: debe ingresar un número entero válido.
Ingrese la superficie mínima en km²: 0
Error: debe ingresar un número mayor a cero.
Ingrese la superficie mínima en km²: -4
Error: debe ingresar un número mayor a cero.
Ingrese la superficie mínima en km²: 435
Ingrese la superficie máxima en km²: 4234234234

----- LISTADO DE PAÍSES -----
Nombre: Argentina
Población: 432423
Superficie: 432432 km²
Continente: America
-----

```

*Figura 7c. Filtro de países por rango de superficie*

## 8.5 Ordenamiento

El sistema permite ordenar la información por nombre, población o superficie. En la prueba se seleccionó el ordenamiento por población en forma descendente, mostrando primero los países con mayor cantidad de habitantes.

```

----- MENÚ PRINCIPAL -----
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar país por nombre
5. Filtrar países por continente
6. Filtrar países por rango de población
7. Filtrar países por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
0. Salir
Ingrese una opción: 8

----- ORDENAR PAÍSES -----
1. Ordenar por nombre
2. Ordenar por población
3. Ordenar por superficie
Seleccione una opción: 2

Tipo de orden:
1. Ascendente
2. Descendente
Seleccione una opción: 2

----- LISTADO DE PAÍSES -----
Nombre: China
Población: 645756756
Superficie: 4234543 km²
Continente: Asia
-----
Nombre: Argentina
Población: 432423
Superficie: 432432 km²
Continente: America
-----
Nombre: Chile
Población: 53454
Superficie: 645645 km²
Continente: America

```

Figura 8. Ordenamiento de países por población en forma descendente. Datos cargados no oficiales

## 8.6 Estadísticas

La opción de estadísticas resume la información principal del dataset: mayor y menor población, promedio de población, promedio de superficie y cantidad de países por continente.

```

----- MENÚ PRINCIPAL -----
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie de un país
4. Buscar país por nombre
5. Filtrar países por continente
6. Filtrar países por rango de población
7. Filtrar países por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
0. Salir
Ingrese una opción: 9

----- ESTADÍSTICAS -----

País con mayor población:
Nombre: China
Población: 645756756
Superficie: 4234543 km²
Continente: Asia
-----

País con menor población:
Nombre: Chile
Población: 53454
Superficie: 645645 km²
Continente: America
-----

Promedio de población: 215414211.00
Promedio de superficie: 1770873.33 km²

Cantidad de países por continente:
America: 2
Asia: 1

```

*Figura 9. Estadísticas generales calculadas por el programa.*

## 8.7 Validación de entradas

También se probaron entradas inválidas, que se pueden observar en varias de las imágenes ya cargadas, como texto en campos numéricos y rangos incorrectos. El sistema responde con mensajes de error claros y vuelve a solicitar los datos.

## 9. Dificultades encontradas

Durante el desarrollo surgieron algunas dificultades técnicas relacionadas con la lectura del archivo CSV, la conversión de datos numéricos, la validación de entradas y la organización del programa en módulos.

Una de las primeras decisiones fue definir cómo representar cada país. Se optó por una lista de diccionarios porque permite mantener la relación entre los campos de cada registro y recorrer fácilmente el conjunto completo para realizar búsquedas, filtros, ordenamientos y estadísticas.

Otro punto importante fue el manejo del archivo CSV. Como el archivo se entrega inicialmente vacío, pero con el encabezado *nombre,poblacion,superficie,continente*, fue necesario asegurar que el programa pudiera iniciar aunque no existieran países cargados. La función de carga lee el archivo y, si no encuentra registros, devuelve una lista vacía. Luego, cuando el usuario agrega países, los datos se guardan nuevamente en el CSV.

También surgieron ajustes en las validaciones. Inicialmente, el sistema podía aceptar nombres formados solo por números y rechazaba continentes que no estaban escritos exactamente igual a la lista mostrada. Para resolverlo, se incorporó una validación específica para el nombre del país y se mejoró la función de continente para aceptar entradas como AMerica, america.

Otro aspecto corregido fue el ordenamiento. En una primera versión, si el usuario ingresaba una opción inválida para el criterio de ordenamiento, el sistema igualmente solicitaba si el orden debía ser ascendente o descendente. Esta lógica fue ajustada para validar primero el criterio seleccionado y volver al menú en caso de error.

Finalmente, se reorganizó el proyecto en una carpeta *modulos/*, separando las funciones en archivos específicos. Esta decisión permitió cumplir con el requisito de modularización y dejar el código más claro, ordenado y fácil de mantener.

## 10. Conclusiones

El desarrollo del Trabajo Práctico Integrador permitió aplicar de manera conjunta los principales contenidos de Programación 1. A partir de un problema concreto, se trabajó con listas, diccionarios, funciones, condicionales, ciclos, archivos CSV, ordenamientos, validaciones y estadísticas básicas.

La lista de diccionarios resultó una estructura adecuada para representar el dataset de países, ya que permite organizar cada registro con claves descriptivas y recorrer el conjunto completo cuando se necesita buscar, filtrar, ordenar u obtener estadísticas.

La organización del código en módulos permitió mejorar la claridad del proyecto. El archivo `main.py` quedó como punto de entrada del programa, mientras que las funciones auxiliares se distribuyeron en archivos específicos dentro de la carpeta `modulos/`. Esta separación facilita la lectura del código, evita concentrar toda la lógica en un único archivo y permite realizar modificaciones futuras de manera más ordenada.

El uso del archivo CSV permitió incorporar una forma básica de persistencia de datos. Aunque el archivo se entrega inicialmente vacío, conserva su estructura mediante el encabezado y permite guardar los países cargados durante la ejecución del programa.

Además, las validaciones implementadas ayudaron a construir un sistema más robusto para el usuario, contemplando campos vacíos, valores numéricos inválidos, rangos incorrectos y nombres no válidos.

En conclusión, el proyecto permitió integrar teoría y práctica en una aplicación funcional, reforzando la importancia de diseñar soluciones ordenadas, legibles, modularizadas y orientadas a resolver problemas concretos, como así también a conectar y coordinar con el compañero las actividades.

## 11. Bibliografía / Webgrafía

- Downey, A. (2015). Think Python — <https://greenteapress.com/wp/think-python-2e/>
- Sweigart, A. (2020). Automate the Boring Stuff with Python — <https://automatetheboringstuff.com>
- Videos y materiales del aula virtual — Campus UTN TUPAD

## 12. Links del proyecto

Repositorio GitHub: <https://github.com/maidanano/tpi-paises-python>

Video demostrativo: <https://youtu.be/1SiKjbChhOs>

Documentación PDF: <https://github.com/maidanano/tpi-paises-python/tree/master/informe>